

说明

本文主要介绍了 CH32X315 通用 MCU 具有四个 ADC 模块；ADC 可使用级联的方式可以对 4 个 ADC 同时使用，可达到同时采集的目的。此外，CH32X315 具有扫描输出功能，可配合开关芯片完成多个通道全自动自动扫描。

适用范围

适用范围	系列
通用 MCU	CH32X315

目录

说明	i
目录	ii
表格索引	ii
图片索引	ii
第 1 章 ADC 模式介绍	1
1.1 CH32X315 ADC 功能简介	1
1.2 ADC 同步采样	1
1.3 ADC 扫描输出	2
第 2 章 ADC 模式使用	4
2.1 ADC 模式配置	4
历史版本	9
声明	10

表格索引

表 1-1 ADC 引脚分配	错误!未定义书签。
表 1-2 CH448 控制表	错误!未定义书签。

图片索引

图 1-1 多通道同步模式转换	错误!未定义书签。
图 1-2 4 组 ADC 同步采样	2
图 1-3 扫描输出引脚状态	错误!未定义书签。

第1章 ADC 模式介绍

1.1 CH32X315 ADC 功能简介

CH32X315 与通用相比，具有以下优点：

- (a) 具有 4 个 ADC，支持多 ADC 转换模式。
- (b) 每个 ADC 通道均有独立引脚，4 组 ADC 共提供 $4 \times 12 = 48$ 个外部通道采样。
- (c) 具有扫描输出功能，可外加开关芯片实现 ADC 通道扩展。

ADC 引脚分配见下表：

表 1-1 ADC 引脚分配

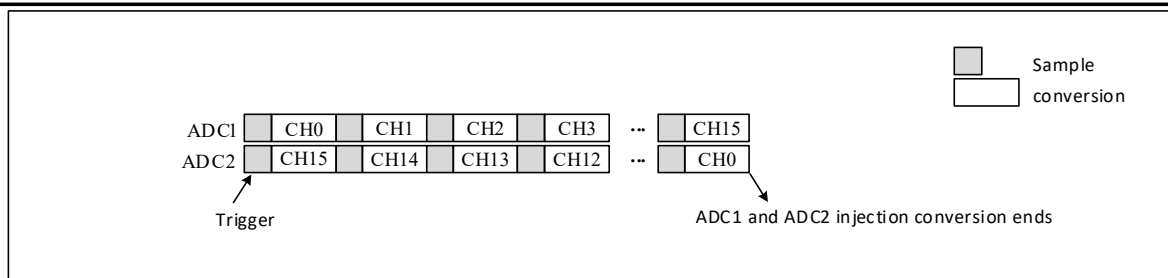
ADC1		ADC2		ADC3		ADC4	
ADC 通道	引脚	ADC 通道	引脚	ADC 通道	引脚	ADC 通道	引脚
ADC1_IN0	PA2	ADC2_IN0	PB6	ADC3_IN0	PB10	ADC4_IN0	PD14
ADC1_IN1	PA3	ADC2_IN1	PB7	ADC3_IN1	PB11	ADC4_IN1	PD15
ADC1_IN2	PA4	ADC2_IN2	PC8	ADC3_IN2	PD8	ADC4_IN2	PC0
ADC1_IN3	PA5	ADC2_IN3	PC9	ADC3_IN3	PD9	ADC4_IN3	PC1
ADC1_IN4	PA6	ADC2_IN4	PC10	ADC3_IN4	PD10	ADC4_IN4	PC2
ADC1_IN5	PA7	ADC2_IN5	PC11	ADC3_IN5	PD11	ADC4_IN5	PC3
ADC1_IN6	PB0	ADC2_IN6	PC12	ADC3_IN6	PD12	ADC4_IN6	PC4
ADC1_IN7	PB1	ADC2_IN7	PC13	ADC3_IN7	PD13	ADC4_IN7	PC5
ADC1_IN8	PB2	ADC2_IN8	PC14	ADC3_IN8	PA8	ADC4_IN8	PC6
ADC1_IN9	PB3	ADC2_IN9	PC15	ADC3_IN9	PA9	ADC4_IN9	PC7
ADC1_IN10	PB4	ADC2_IN10	PB8	ADC3_IN10	PD4	ADC4_IN10	PA0
ADC1_IN11	PB5	ADC2_IN11	PB9	ADC3_IN11	PD5	ADC4_IN11	PA1

此外，4 个 ADC 都支持使用 DMA 操作。

1.2 ADC 同步采样

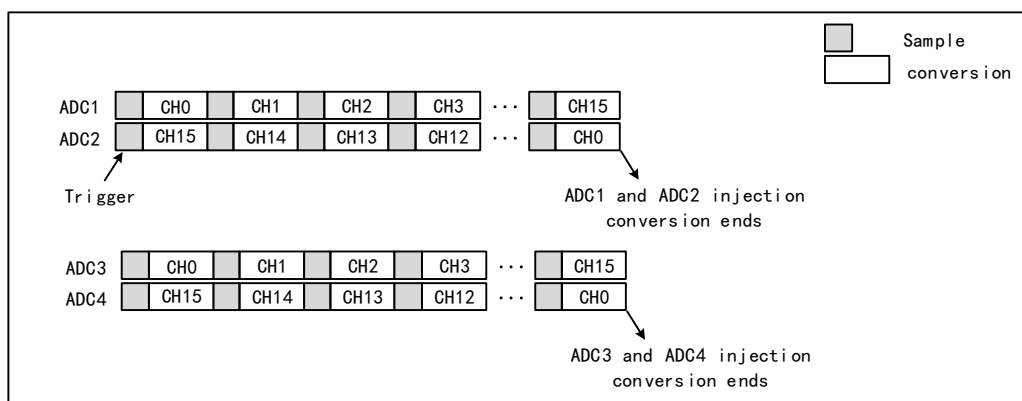
CH32X315 具有 4 个 ADC，可分别支持双 ADC 模式，在双 ADC 模式下可配置同步模式，可对所对应通道进行同步采集。在双 ADC 模式下，只可对 ADC1/ADC2 与 ADC3/ADC4 进行配置，在同步模式下，主 ADC 只需一个触发源，次 ADC 同步进行转换，即完成电压信号同一时刻进行采集。图 1-1 为多通道同步模式转换示意图。

图 1-1 多通道同步模式转换



在双 ADC 模式下，只可对 ADC1/ADC2 与 ADC3/ADC4 进行操作，因此可以与级联的方式进行 4 组 ADC 进行互联。CH32X315 级联可配置前级 ADC，可通过设置前级触发源与触发延时可对多 ADC 进行互联，在使用级联模式下，可与双 ADC 模式下配置，即可完成 4 个 ADC 进行采样。若设置触发延时为 0，在使用一个触发源下，即可完成 4 组 ADC 同步采样。当触发延时为 0，同时使用双 ADC 同步模式时，转换序列为：

图 1-1 4 组 ADC 同步采样

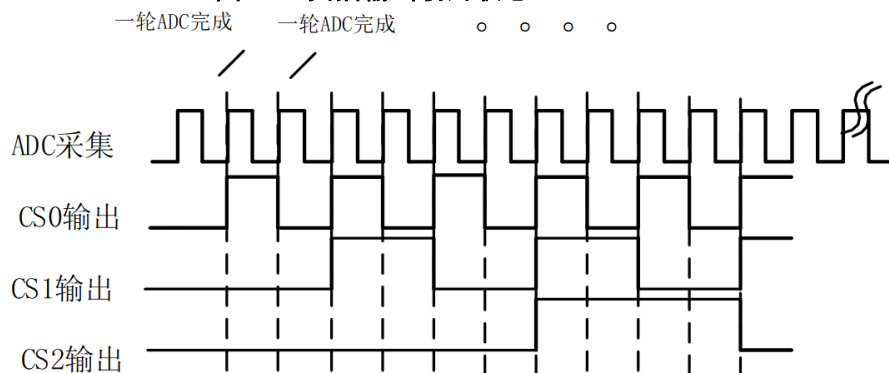


在该模式下，可同时采集四个通道。与通用双 ADC 相比，采集速度可以提升一倍。由于每个 ADC 通道引脚均不互斥，因此 CH32X315 在使用该模式时与其他通用系列相比也没有通道转换序列限制。

1.3 ADC 扫描输出

CH32X315 支持扫描输出模式，该模式下在 ADC 扫描完成后轮次计数后对应的 ADCSx 引脚进行扫描输出。依据 ADC 的配置：在一次触发源开启转换时到最后一个 ADC 序列采集完成后为一轮 ADC 采集完成。若设置为 3 位计数周期时，ADC 采集一轮后对应的引脚输出状态为：

图 1-2 扫描输出引脚状态



基于此功能，可外接 8 通道开关芯片进行 ADC 通道扩展；以选用 CH448 2 路 8 选一开关芯片为例，其控制表为：

表 1-2CH448 控制表

XEN#	YEN#	SEL2	SEL1	SEL0	AX	AY
0		0	0	0	选择 A0X	选择 A0Y
0		0	0	1	选择 A1X	选择 A1Y
0		0	1	0	选择 A2X	选择 A2Y
0		0	1	1	选择 A3X	选择 A3Y
0		1	0	0	选择 A4X	选择 A4Y
0		1	0	1	选择 A5X	选择 A5Y
0		1	1	0	选择 A6X	选择 A6Y
0		1	1	1	选择 A7X	选择 A7Y
1		x	x	x	全部断开	全部断开

可以看出，使用扫描输出模式可以依次完成对 A0X-A7X，A0Y-A7Y 的外部扩展通道依次采集。从而由 2 路 ADC 扩展至 16 路 ADC 通道。

第2章 ADC 模式使用

2.1 ADC 模式配置

1.同步规则模式

在使用 4 个 ADC 同步采样时，首先需要配置双 ADC 同步模式，需要对 ADC1，ADC2 双 ADC 模式选择同步模式，示例已规则同步为例，此模式下用于转换规则通道序列，当 ADC1 触发转换时，同时它也将用于同步触发 ADC2。转换完成时，将产生一个 32 位 DMA 传输请求。同样 ADC3 与 ADC4 也有相同的效果。

图 2-1 同步规则模式选择

```
ADC_InitStructure.ADC_Mode = ADC_Mode_RegSimult;
ADC_InitStructure.ADC_ScanConvMode = DISABLE;
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfChannel = 1;
ADC_Init(ADC1, &ADC_InitStructure);
ADC_Init(ADC2, &ADC_InitStructure);
```

选择同步规则模式

2.级联模式

配置 ADC1/ADC2 与 ADC3/ADC4 双 ADC 规则同步模式后，可使用级联模式达到一个触发源开启四个 ADC 转换。级联模式需要配置：

(1) 选择前级 ADC，根据 ADC 前级配置可选择 ADC 模块转换序列；如 ADC3 的前级配置为 ADC1，则只需启动 ADC1 触发源，根据配置 ADC3 进行转换，不需要启动 ADC3 的触发源。

(2) 选择级联使能；根据配置注入组或规则组触发选择级联使能为注入组或规则组；选择规则级联使能时，前级 ADC 规则组触发后，后级 ADC 规则组进行触发转换。

(3) 设置触发延时；可配置规则/注入通道触发延时，设置步进为一个 ADC 时钟周期，如设置触发延时为 0，可达到前级 ADC 触发转换时，后级 ADC 同时进行转换；可达到同步采集的效果。在使用触发延时；ADC 的延迟设置配置的是后级的 ADC 的延迟 当前 ADC 没有后级不配置。

在该功能下，通过设置触发延时为 0，使用双 ADC 同步采集模式下，可完成 4 个 ADC 同步采集；在扫描模式下，转换序列转换周期需要相同用于采集同一时刻信号源。

已规则同步为例，若使用 4 个 ADC 同步采样时，需要开启 ADC3 规则级联使能；前级 ADC 为 ADC1，并 ADC1 触发延时为 0。

图 2-2ADC 级联模式配置

```
ADC-RegularChannelCascadeCmd(ADC3, ENABLE);
ADC_CascadeContsFrontConfig(ADC3, ADC_CONTS_Front_ADC1);
ADC-RegularChannelCascadeDLYConfig(ADC1, 0x00);
```

3.扫描输出

ADC 扫描输出模式引脚可外接开关芯片可外扩 ADC 通道扩展，并实现 ADC 通道自动切换。ADC 扫描输出有 3 个引脚输出，因此可外接 CH448 为 8 选一开关芯片配合，进行 2*8 个通道自动切换。扫描输出引脚为

表 2-1 扫描输出引脚

ADC 扫描功能	ADCS0	ADCS1	ADCS2
----------	-------	-------	-------

可选引脚

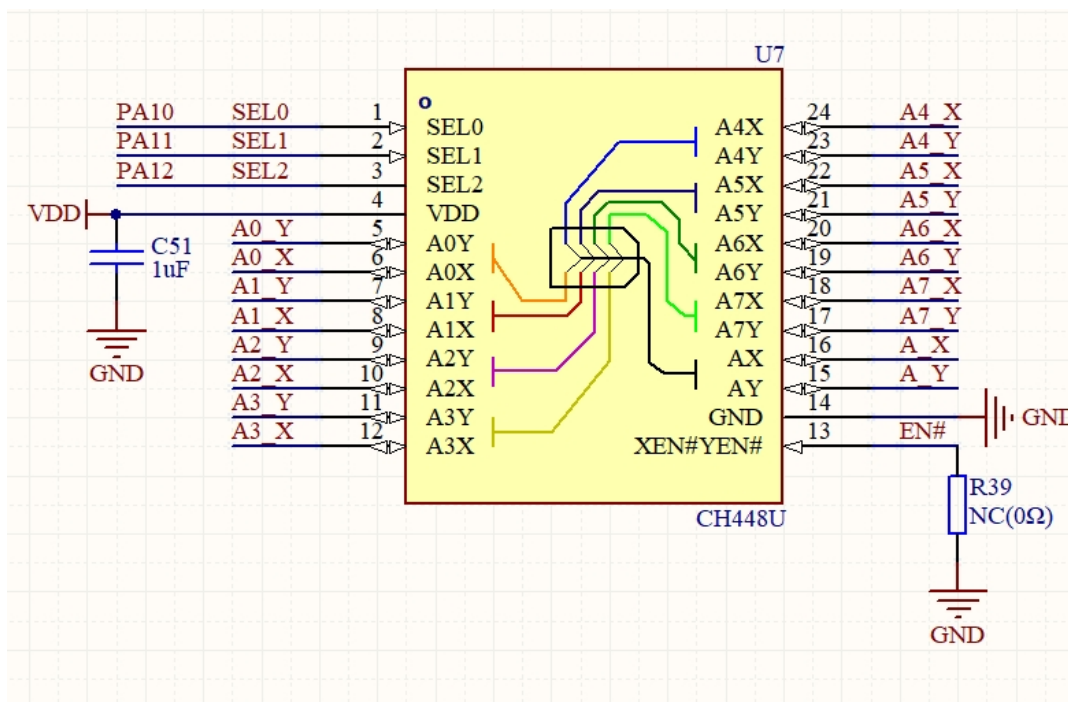
PA10 (AF5)

PA11 (AF5)

PA12 (AF5)

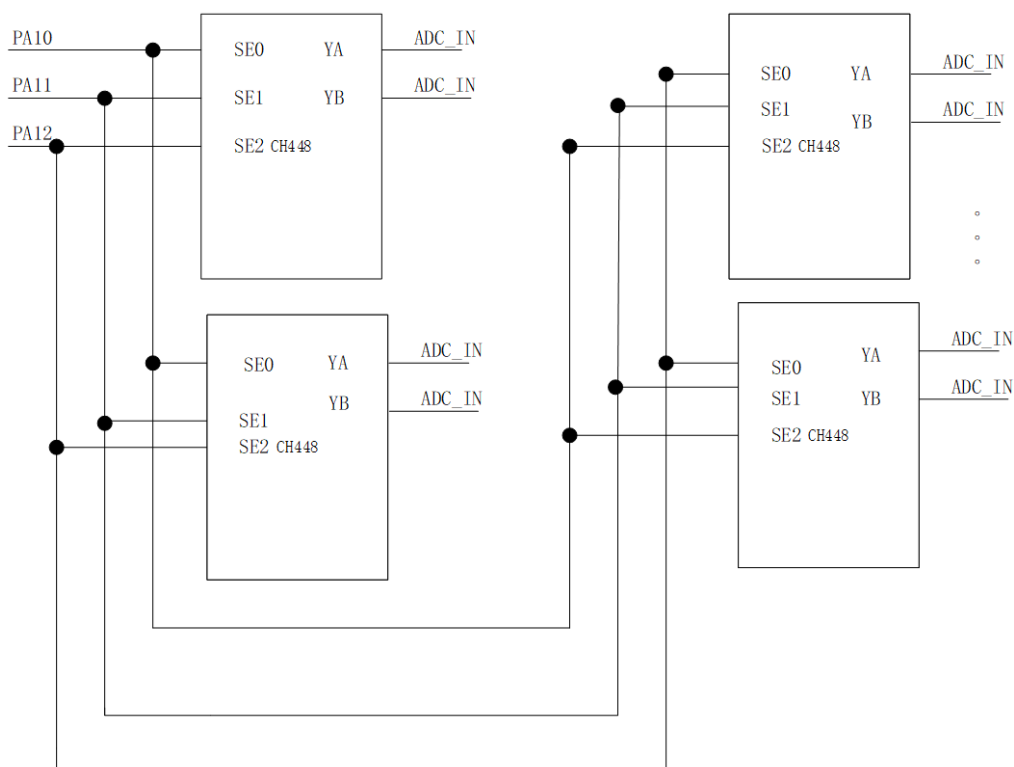
依据开关芯片控制表，只需将开关芯片开关的选择输入端 SEL0-SEL2 与 ADC 的扫描输出端 ADCS0- ADCS2 短接，在每次 ADC 转换一轮后，ADC 输入端 AX/AY 可以自动切换模拟开关的输入选择端 A0X-A7X 与 A0Y-A7Y。

图 2-3CH448 示意图



基于图，可外扩多个 8 选一的开关芯片，完成对 ADC 通道扩展，支持最多 $8 \times 48 = 384$ 个 ADC 通道全自动切换。

图 2-4CH448 多通道开关拓扑图



在使用 CH32X315 扫描输出时，需要配置步骤如下；

(1) 配置扫描输出配置源；通过 ADCx_SCAN_CNT 寄存器 SCSCFG 位选择规则通道或注入通道进行扫描通道源选择；来确认规则通道或者注入通道采集一轮后，ADCSx 引脚输出发生变化。

(2) 配置扫描周期计数使能位，过 ADCx_SCAN_CNT 寄存器使能 SCANOEx 位进行周期计数使能位使能，根据选择外接开关器件来配置周期计数使能位；若使用 8 选 1 开关 CH448 配置扫描周期计数使能为 SCANOEO- SCANOE2；若选用 4 选 1 开关 CH444 可配置扫描周期计数使能为 SCANOEO- SCANOE1。

(3) 设置扫描周期计数器输出 ADC 选择；通过设置 R32_EXTEN_CTR 寄存器 ADC_SCAN_SEL[1:0] 位选择 ADCx 扫描周期计数输出至 GPIO。若选择为 ADC1 时，只有当 ADC1 一轮采集结束后，输出扫描引脚电平才会发生改变。

设置为规则组为配置扫描输出，扫描计数周期使能为 SCANOEO- SCANOE2，设置扫描周期计数为 ADC1 有效。

图 2-5 扫描输出配置

```
GPIO_PinAFConfig(GPIOA, GPIO_PinSource10, GPIO_AF5);
GPIO_PinAFConfig(GPIOA, GPIO_PinSource11, GPIO_AF5);
GPIO_PinAFConfig(GPIOA, GPIO_PinSource12, GPIO_AF5);
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10|GPIO_Pin_11|GPIO_Pin_12;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_Very_High;
GPIO_Init(GPIOA, &GPIO_InitStructure);

ADC_ScanSelConfig(ADC1_SCAN_OUT);
ADC_ScanModeConfig(ADC1, ADC_ScanCH_SRC_Regular);
ADC_ScanCountOutputConfig(ADC1, ADC_ScanCntOutMode7);
```

4. 整体配置

实现效果为，4 个 ADC 同步采样，外接 2 个 8 选 1 开关芯片实现 4*8 个外扩 ADC 通道，定时轮询采集；ADC 初始化为：

图 2-6 ADC 初始化

```
void ADC_Function_Init(void)
{
    ADC_InitTypeDef ADC_InitStructure={0};
    GPIO_InitTypeDef GPIO_InitStructure={0};

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA|RCC_APB2Periph_GPIOB, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1|RCC_APB2Periph_ADC2, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC3|RCC_APB2Periph_ADC4, ENABLE);
    RCC_ADCCLKConfig(RCC_ADCPRE_Div2, RCC_PPRE2_Div8);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    GPIO_PinAFConfig(GPIOA, GPIO_PinSource10, GPIO_AF5);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource11, GPIO_AF5);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource12, GPIO_AF5);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10|GPIO_Pin_11|GPIO_Pin_12;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_Very_High;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    ADC_DeInit(ADC1);
    ADC_DeInit(ADC2);
    ADC_DeInit(ADC3);
    ADC_DeInit(ADC4);
    ADC_InitStructure.ADC_Mode = ADC_Mode_Regular;
    ADC_InitStructure.ADC_ScanConvMode = DISABLE;
    ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
    ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_Rising;
    ADC_InitStructure.ADC_NbrOfChannels = 1;
    ADC_Init(ADC1, &ADC_InitStructure);

    ADC_ScanSelConfig(ADC1_SCAN_OUT);
    ADC_ScanModeConfig(ADC1, ADC_ScanCH_SRC_Regular);
    ADC_ScanCountOutputConfig(ADC1, ADC_ScanCntOutMode7);

    ADC_SampleTimeNodeConfig(ADC1, ADC_Channel_1, ADC_SampleTime_Node0);
    ADC_ReguilarChannelConfig(ADC1, ADC_Channel_1, 1);
    ADC_SetModeSampleTimeConfig(ADC1, ADC_Channel_1, ADC_SampleTime_CyclesNode7);
    ADC_ExternalTrigConvCmd(ADC1, ENABLE);
    ADC_Cmd(ADC1, ENABLE);

    ADC_BufferCmd(ADC1, DISABLE); //disable buffer
    ADC_ResetCalibration(ADC1);
    while(ADC_GetResetCalibrationStatus(ADC1));
    ADC_StartCalibration(ADC1);
    while(ADC_GetCalibrationStatus(ADC1));

    ADC_Init(ADC2, &ADC_InitStructure);
    ADC_SampleTimeNodeConfig(ADC2, ADC_Channel_0, ADC_SampleTime_Node0);
    ADC_ReguilarChannelConfig(ADC2, ADC_Channel_0, 1);
    ADC_SetModeSampleTimeConfig(ADC2, ADC_Channel_0, ADC_SampleTime_CyclesNode7);
    ADC_ExternalTrigConvCmd(ADC2, ENABLE);
    ADC_Cmd(ADC2, ENABLE);

    ADC_BufferCmd(ADC2, DISABLE); //disable buffer
    ADC_ResetCalibration(ADC2);
    while(ADC_GetResetCalibrationStatus(ADC2));
    ADC_StartCalibration(ADC2);
    while(ADC_GetCalibrationStatus(ADC2));

    ADC_Init(ADC3, &ADC_InitStructure);
    ADC_ReguilarChannelCascadedCmd(ADC3, ENABLE);
    ADC_CascadeConvFrontConfig(ADC3, ADC_CONTS_Front_ADC1);
    ADC_SampleTimeNodeConfig(ADC3, ADC_Channel_0, ADC_SampleTime_Node0);
    ADC_ReguilarChannelConfig(ADC3, ADC_Channel_0, 1);
    ADC_SetModeSampleTimeConfig(ADC3, ADC_Channel_0, ADC_SampleTime_CyclesNode7);

    ADC_ExternalTrigConvCmd(ADC3, ENABLE);
    ADC_Cmd(ADC3, ENABLE);

    ADC_BufferCmd(ADC3, DISABLE); //disable buffer
    ADC_ResetCalibration(ADC3);
    while(ADC_GetResetCalibrationStatus(ADC3));
    ADC_StartCalibration(ADC3);
    while(ADC_GetCalibrationStatus(ADC3));

    ADC_Init(ADC4, &ADC_InitStructure);
    ADC_SampleTimeNodeConfig(ADC4, ADC_Channel_0, ADC_SampleTime_Node0);
    ADC_ReguilarChannelConfig(ADC4, ADC_Channel_0, 1);
    ADC_SetModeSampleTimeConfig(ADC4, ADC_Channel_0, ADC_SampleTime_CyclesNode7);
    ADC_ExternalTrigConvCmd(ADC4, ENABLE);
    ADC_Cmd(ADC4, ENABLE);

    ADC_BufferCmd(ADC4, DISABLE); //disable buffer
    ADC_ResetCalibration(ADC4);
    while(ADC_GetResetCalibrationStatus(ADC4));
    ADC_StartCalibration(ADC4);
    while(ADC_GetCalibrationStatus(ADC4));
}
```

选择输出扫描引脚为 ADCS0-ADCS2，扫描周期计数器输出为 ADC1 有效，配置为双 ADC 同步规则配置，使用级联为 ADC3 前级为 ADC1 且触发延时为 0。选择外部触发源为 TIM1_CC1，进行定时触发；分别配置 ADC1-ADC4 同序列通道相同采样周期。

DMA 初始化：在双 ADC 同步规则模式下，转换完成时，将产生一个 32 位 DMA 传输请求，其中 ADC1 高低 16 位分别为 ADC2 与 ADC1 采集数值；ADC3 高低 16 位分别为 ADC4 与 ADC3 采集数

值。因此配置 DMA 初始化时只需使能 ADC1 与 ADC3 通道。

图 2-7DMA 初始化

```

1 void DMA_Tx_Init( DMA_Channel_TypeDef* DMA_CHx, u32 ppadr, u32 memadr, u16 bufsize)
2 {
3     DMA_InitTypeDef DMA_InitStructure={0};
4
5     RCC_HBPeriphClockCmd( RCC_HBPeriph_DMA1, ENABLE );
6
7     DMA_DeInit(DMA_CHx);
8     DMA_InitStructure.DMA_PeripheralBaseAddr = ppadr;
9     DMA_InitStructure.DMA_Memory0BaseAddr = memadr;
10    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralSRC;
11    DMA_InitStructure.DMA_BufferSize = bufsize;
12    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
13    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
14    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
15    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
16    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
17    DMA_InitStructure.DMA_Priority = DMA_Priority_VeryHigh;
18    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
19    DMA_Init( DMA_CHx, &DMA_InitStructure );
20 }
21
22 DMA_Tx_Init( DMA1_Channel1, (u32)&ADC1->RDATAR, (u32)TxBuf1, 1 );
23 DMA_Cmd( DMA1_Channel1, ENABLE );
24 DMA_Tx_Init( DMA1_Channel9, (u32)&ADC3->RDATAR, (u32)TxBuf2, 1 );
25 DMA_Cmd( DMA1_Channel9, ENABLE );
26 NVIC_EnableIRQ(DMA1_Channel1_IRQn);
27 DMA_ITConfig( DMA1_Channel1, DMA_IT_TC, ENABLE );
28 NVIC_EnableIRQ(DMA1_Channel9_IRQn);
29 NVIC_SetPriority(DMA1_Channel9_IRQn,0x70);
30 DMA_ITConfig( DMA1_Channel9, DMA_IT_TC, ENABLE );

```

触发信号配置：选用定时器比较触发，用于定时采样。

图 2-8 定时器初始化

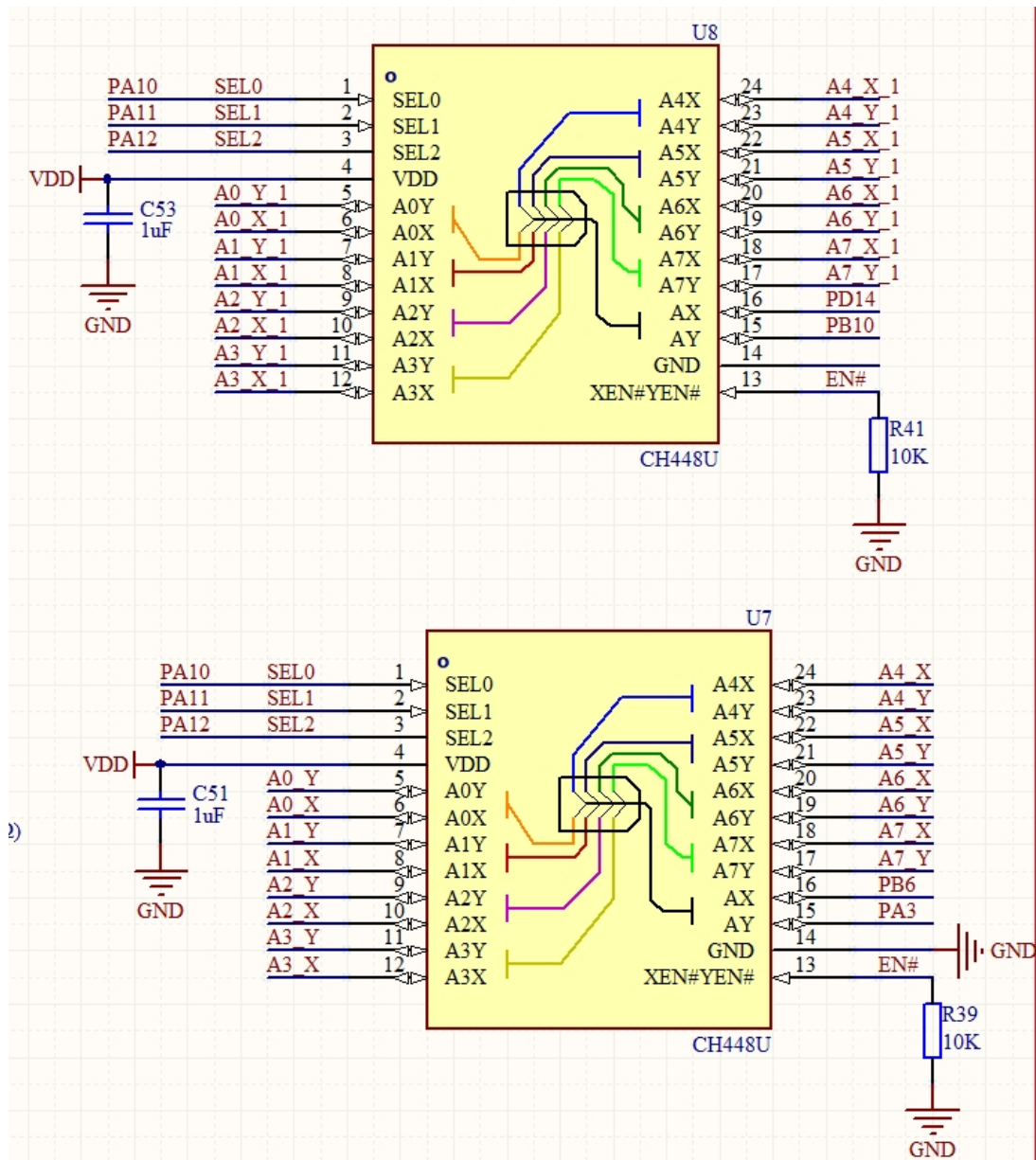
```

1 void TIM1_PWM_In( u16 arr, u16 psc, u16 ccp )
2 {
3
4     TIM_OCInitTypeDef TIM_OCInitStructure={0};
5     TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure={0};
6
7     RCC_HB2PeriphClockCmd( RCC_HB2Periph_GPIOA | RCC_HB2Periph_TIM1, ENABLE );
8
9     TIM_TimeBaseInitStructure.TIM_Period = arr;
10    TIM_TimeBaseInitStructure.TIM_Prescaler = psc;
11    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
12    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
13    TIM_TimeBaseInit( TIM1, &TIM_TimeBaseInitStructure);
14
15    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
16    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
17    TIM_OCInitStructure.TIM_Pulse = ccp;
18    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;
19    TIM_OC1Init( TIM1, &TIM_OCInitStructure );
20
21    TIM_CtrlPWMOutputs(TIM1, ENABLE );
22    TIM_OC1PreloadConfig( TIM1, TIM_OCPreload_Disable );
23    TIM_ARRPreloadConfig( TIM1, ENABLE );
24    TIM_SelectOutputTrigger( TIM1, TIM_TRGOSource_Update );
25    TIM_Cmd( TIM1, ENABLE );
26 }

```

按照选择 ADC 通道，开关芯片 X/Y 公共端分别接入 ADC2/ADC1 与 ADC3/ADC4 的 ADC 输入通道；开关输入端口 AxX 与 AxY 分别接入被测信号，通过配置即可完成对 ADC1 通道 1 对 A0Y-A7Y 定时器轮询采样；ADC2 通道 0 对 A0X-A7X 定时器轮询采样；ADC3 通道 0 对 A0Y_1-A7Y_1 定时器轮询采样；ADC4 通道 0 对 A0X_1-A7X_1 定时器轮询采样；外接开关芯片电路图：

图 2-9CH448 外围电路



历史版本

更新内容

日期	版本	变更内容
2025/11/20	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。